

Data Structures & Algorithms (DSA) Test — Set B

Difficulty: Medium Level

Duration: 75 Minutes

Total Questions: 5

Important Instructions:

- Solutions must comply strictly with the recommended optimal runtime and space complexities.
- In-place algorithms are required where explicitly requested; auxiliary structures are banned in those cases.
- Ensure all solution scripts contain comments detailing structural approaches and time/space constraints.

Question 1 — Pattern (Medium)

Write an application that prints an inverted-and-upright combined **Number Hourglass Pattern** for a peak initial count value of $n = 4$.

Expected Output:

```
4 4 4 4 4 4 4
 3 3 3 3 3
  2 2 2
   1
  2 2 2
 3 3 3 3 3
4 4 4 4 4 4 4
```

- **Input:** An integer n tracking the boundaries.
- **Output:** Direct visual matrix representation using spaces for accurate alignment.

Question 2 — String (Medium)

Given a string s , identify and return the length of the **longest contiguous substring** that contains entirely unique characters (no structural repetitions).

- **Example Input:** $s = \text{"abcabcbb"}$
- **Example Output:** 3
- **Explanation:** The optimal substring answer is "abc", keeping its sequence length locked at 3.

- **Constraints:** $0 \leq s.length() \leq 5 \times 10^4$. Inputs are composed of alphanumeric standard symbols and literal white space values.

Question 3 — Hashing (Medium)

Given an unsorted array of integers `nums`, track down and return the total length of the **longest consecutive elements sequence** found anywhere within the parameters. You must structure an algorithm that finishes within optimal $O(n)$ processing runtimes.

- **Example Input:** `nums = [100, 4, 200, 1, 3, 2]`
- **Example Output:** 4
- **Explanation:** The longest sorted continuous sequence is [1, 2, 3, 4], marking an output scale of 4.
- **Constraints:** $0 \leq nums.length \leq 10^5$; $-10^9 \leq nums[i] \leq 10^9$.

Question 4 — Array (Medium)

Given an integer array `nums` sorted cleanly in non-decreasing order, cyclically rotate your overall array space to the right side by k sequence steps (where k is non-negative). Optimize your implementation to run strictly ****in-place**** using $O(1)$ auxiliary memory.

- **Example Input:** `nums = [1, 2, 3, 4, 5, 6, 7], k = 3`
- **Example Output:** [5, 6, 7, 1, 2, 3, 4]
- **Constraints:** $1 \leq nums.length \leq 10^5$; $-2^{31} \leq nums[i] \leq 2^{31} - 1$; $0 \leq k \leq 10^5$.

Question 5 — Matrix (Medium)

Given an $m \times n$ integer matrix, if any unique element resolves to a value of 0, sweep and overwrite its entire row grid and column grid to contain all 0 items. This mapping operation must be finalized ****in-place**** utilizing optimal $O(1)$ additional auxiliary indicator variables.

Example Input:

```
[[1, 1, 1],  
 [1, 0, 1],  
 [1, 1, 1]]
```

Expected Output:

```
[[1, 0, 1],  
 [0, 0, 0],  
 [1, 0, 1]]
```

- **Constraints:** $m == \text{matrix.length}$; $n == \text{matrix}[0].length$; $1 \leq m, n \leq 200$; $-2^{31} \leq \text{matrix}[i][j] \leq 2^{31} - 1$.